

LLMs as Graph Reasoners: Edge Criticality and Path Robustness in Last-Mile Routing

Amazon TSP Project

Amogh Lal (23115012), Sargam Goyal (23125031), Tanish Verma (23125035)

May 4, 2026

Abstract

The Traveling Salesman Problem (TSP) is a fundamental combinatorial optimisation challenge with applications in last-mile delivery and routing. This paper presents an evaluation framework for quantifying *edge importance* in TSP solutions—a measure of how critical an edge is to the overall tour cost. We introduce an evaluation method, a ground-truth importance metric computed by systematically perturbing each edge in the base optimal tour and measuring the relative increase in tour cost. We implement the framework using the LKH-3 metaheuristic solver with parallel edge perturbation processing. Our results demonstrate the feasibility of computing exact edge importance for large real-world routing instances. Building on this, we present a detailed experimental plan for evaluating large language models (LLMs) as edge importance predictors against this benchmark, and assess their downstream impact on robust route optimisation via cost-perturbation re-routing.

1 Goal

The overarching goal of this project is to establish a robust framework for understanding and predicting edge importance in TSP solutions within the context of last-mile delivery optimisation. Specifically, we aim to:

1. **Compute Ground-Truth Importance:** Define and compute a rigorous metric for quantifying how critical each edge is to the optimality of a TSP tour.
2. **Enable Predictor Benchmarking:** Create a benchmark against which learning-based importance predictors (e.g., LLMs, neural networks) can be evaluated.
3. **Support Cost Perturbation Routing:** Use predicted importance scores to modify edge costs and assess the impact on routing quality.

The motivation is to enable data-driven optimisation systems where machine learning models learn to predict which edges are most critical, allowing for intelligent route modification or graph construction.

2 Data Description

2.1 Dataset Overview

This project uses the **2021 Amazon Last Mile Routing Research Challenge Dataset**, a publicly available dataset released by Amazon.com in collaboration with the MIT Center for Transportation and Logistics [1]. The dataset is hosted on AWS S3 (`s3://amazon-last-mile-challenges`) and is accessible without an AWS account. It is released under a Creative Commons Attribution-NonCommercial 4.0 International licence.

The dataset contains route-, stop-, and package-level features for **9,184 historical delivery routes** executed by Amazon drivers in 2018, covering five metropolitan areas in the United States. All personally identifiable information has been removed: route and package identifiers were randomly regenerated and GPS coordinates were obfuscated to preserve anonymity.

The data is split into two subsets:

- **Training set:** 6,112 historical routes (`almrrc2021_data_training/`).
- **Evaluation set:** 3,072 historical routes (`almrrc2021_data_evaluation/`).

2.2 Data Structure

Each route is identified by a unique **RouteID** (UUID format). The dataset is organised into the following JSON files per split:

Route-level data (`route_data.json`). Contains metadata for each route:

- **station_code:** alphanumeric identifier for the depot/delivery station from which the route departed.
- **date_YYYY_MM_DD:** calendar date of departure.
- **departure_time_utc:** UTC departure timestamp.
- **executor_capacity_cm³:** volumetric capacity of the delivery vehicle in cm³.
- **route_score:** a categorical quality label for the observed stop sequence, reflecting driver execution quality (values: `High`, `Medium`, `Low`).

Stop-level data (`route_data.json`). Each route contains a list of stops, each with:

- **lat, lng:** WGS 84 latitude and longitude coordinates.
- **type:** indicates whether the stop is the originating station or a delivery location.
- One stop per route is designated the depot (station); all others are customer delivery locations.

Package-level data (`package_data.json`). Packages are associated with stops and carry:

- **scan_status:** delivery outcome (e.g., `delivered`, `attempted`).
- **time_window:** optional delivery window with `start_time_utc` and `end_time_utc`.
- **planned_service_time_seconds:** estimated dwell time at the stop.

- **dimensions:** package depth, height, and width in centimetres.

Travel times (`travel_times.json`). Provides a stop-to-stop travel-time matrix (in seconds) for each route, derived from map data. These matrices are used to construct the distance graphs on which the TSP is solved.

Actual sequences (`actual_sequences.json`). Records the stop-visit order actually executed by the driver, encoded as a mapping from stop identifiers to sequence positions. These observed sequences serve as the behavioural ground truth for sequence-learning models in the original challenge.

Evaluation scores (`eval_score.json`). A per-route scalar score measuring how closely a proposed stop sequence matches the driver’s actual sequence, used as the primary metric in the original challenge.

2.3 Relevance to This Work

For our edge-importance framework, each route instance is treated as a symmetric TSP on a graph whose nodes are delivery stops (including the depot) and whose edge weights are derived from the travel-time matrix. We apply 4-nearest-neighbour sparsification to keep the graph tractable. The `route_score` field allows us to filter for high-quality driver sequences, ensuring that the base TSP solution reflects expert routing behaviour. Because the dataset spans multiple cities and route sizes, it also provides a natural testbed for the generalisation experiments outlined in Section 6.3.

3 Proposed Idea

3.1 Edge Importance Definition

We define edge importance as the relative increase in optimal tour cost when an edge is removed or heavily penalised:

$$I_e = \frac{C(G \setminus e) - C(G)}{C(G)} \quad (1)$$

where:

- G is the distance graph with edge weights corresponding to travel distances,
- e is a single edge in the base optimal tour,
- $C(G)$ is the optimal tour cost on the original graph,
- $C(G \setminus e)$ is the optimal tour cost when edge e is removed (simulated via a large penalty).

A higher importance value indicates that the edge is more critical to maintaining tour optimality. Edges with high importance are those for which suitable alternatives are limited.

3.2 Computational Framework

The framework follows a multi-step pipeline:

1. **Graph Construction:** Build a sparse K -nearest neighbours graph ($K = 4$) from delivery locations to ensure connectivity while reducing problem size.

2. **Base TSP Solution:** Solve the TSP on the original graph using LKH-3 to obtain base cost $C(G)$ and base route R_{base} .
3. **Parallel Edge Perturbation:** For each edge in R_{base} , create a perturbed graph by setting the edge weight to a large penalty $M = \alpha \cdot \max(c)$ and re-solve the TSP.
4. **Importance Computation:** Calculate edge importance using Equation (1).
5. **Cost Modification & Routing:** Modify edge costs based on predicted importance and re-solve TSP to measure routing impact.

3.3 Key Design Decisions

- **Penalty Over Removal:** We use a finite penalty rather than edge removal to maintain graph connectivity and avoid numerical instabilities in the solver.
- **Warm-Start Optimisation:** Warm-starting from the base solution reduces convergence time from hundreds to approximately 100 solver trials per edge.
- **Parallel Execution:** Edge perturbations are embarrassingly parallel—we use a `ProcessPoolExecutor` to solve multiple perturbed instances concurrently.
- **FULL MATRIX Format:** Distance matrices already encode missing edges as large finite values, enabling direct representation without edge list conversion.

4 Methodology

4.1 Algorithm

Algorithm 1 Evaluation Method: Edge Importance Computation

```

1: procedure COMPUTEEDGEIMPORTANCE(matrix, baseRoute, baseCost, baseTourFile)
2:    $n \leftarrow \text{len}(\text{baseRoute})$ 
3:    $\text{penalty} \leftarrow 7 \times \max(\text{matrix})$ 
4:    $\text{edges} \leftarrow \{\text{tour edges from baseRoute}\}$ 
5:    $\text{importance} \leftarrow \{\}$  ▷ Dictionary
6:   for each edge  $(u, v) \in \text{edges}$  (in parallel) do
7:      $\text{perturbed} \leftarrow \text{copy}(\text{matrix})$ 
8:      $\text{perturbed}[u][v] \leftarrow \text{penalty}$ 
9:      $\text{perturbed}[v][u] \leftarrow \text{penalty}$ 
10:     $C_{\text{perturbed}} \leftarrow \text{SolveTSP}(\text{perturbed}, \text{baseTourFile})$ 
11:     $I_e \leftarrow (C_{\text{perturbed}} - \text{baseCost}) / \text{baseCost}$ 
12:     $\text{importance}[(u, v)] \leftarrow I_e$ 
13:   end for
14:   return importance
15: end procedure

```

4.2 Implementation Details

Solver Configuration.

- Solver: LKH-3 (Lin–Kernighan–Helsgaun algorithm).

- Runs per instance: 1.
- Max trials per run: 100.
- Penalty multiplier: $7\times$ (balances dominance and numerical stability).

Parallelisation.

- Workers: CPU count minus 1 (reserves one core for OS).
- Execution model: `ProcessPoolExecutor` with asynchronous job completion tracking.
- Memory optimisation: Shallow row-wise copying to avoid full matrix duplication.

Warm-Start Strategy. Each perturbed solve uses the base tour as an initial solution, dramatically reducing solver iterations. This is crucial for computational tractability when processing dozens of edges per route.

5 Results

5.1 Test Instance

We evaluated the framework on a real-world Amazon last-mile routing instance:

- Route ID: `RouteID 00143bdd-0a6b-49ec-bb35-36593d303e77`
- Number of locations: 119 nodes
- Graph type: 4-nearest neighbours sparsified distance graph

5.2 Base Solution

Metric	Value
Base Tour Cost	8088.40
Base Tour Length (edges)	119

Table 1: Base TSP solution metrics for the test instance.

5.3 Edge Importance Distribution

The framework successfully computed importance scores for all 119 edges in the base route. Key observations:

- **Top-10 Critical Edges:** Importance values ranged from 0.050 to 0.068, representing a 5–6.8% increase in tour cost when perturbed.
- **Mid-Importance Edges:** The majority of edges showed importance in the range $[0.010, 0.040]$.
- **Low-Importance Edges:** Approximately 15% of edges exhibited importance below 0.005, indicating good alternative paths exist.

The computed edge importance was serialised to JSON format, enabling downstream analysis and comparison with predictor estimates.

5.4 Computational Performance

- Base TSP solve time: $\approx 2\text{--}5$ seconds.
- Total evaluation runtime: $\approx 3\text{--}5$ minutes for a 119-node instance (parallel processing across 7 workers).
- Speedup from warm-starting: approximately $3\times$ reduction in solver trials compared to cold starts.

The parallelisation proved effective, with near-linear scaling up to the number of available CPU cores.

6 Roadmap

Having established the ground-truth evaluation framework and validated it on a real-world routing instance, the natural next step is to build out the full pipeline described in the project outline. We organise the remaining work into four tightly coupled stages: *predictor development*, *routing integration*, *evaluation*, and *scaling & long-term extensions*.

6.1 Predictor Development

6.1.1 LLM-Based Importance Predictor

The central near-term goal is to determine whether a large language model can approximate ground-truth importance scores given only the edge list and associated weights. The required steps are:

1. **Prompt design.** Craft a structured prompt that presents the edge list (node pairs and distances) together with optional in-context examples, and instructs the model to output a numerical importance score $\hat{I}_e$ for each edge. Particular care is needed to avoid positional bias and to keep token count manageable for large instances.
2. **Output parser.** Implement a robust parser that extracts $\hat{I}_e$ values from free-form model output, handles refusals or malformed responses gracefully, and re-ranks ties deterministically.
3. **Few-shot examples.** Investigate how many labelled examples (from prior prior evaluation runs) are required in-context before the LLM’s Spearman correlation with ground truth saturates.
4. **Normalisation.** Because absolute output scales vary across model families and prompt phrasings, apply sum-normalisation

$$\hat{I}_e \leftarrow \frac{\hat{I}_e}{\sum_{e'} \hat{I}_{e'}}$$

before any metric computation, mirroring the treatment of ground-truth scores.

5. **Alpha tuning.** The cost-modification weight $\alpha \in [0.2, 1.0]$ controls how strongly predicted importance inflates edge costs:

$$c'_e = c_e(1 + \alpha \hat{I}_e).$$

α should be treated as a hyper-parameter and tuned on a held-out set of routes to maximise routing quality rather than raw correlation.

6.2 Routing Integration

6.2.1 Cost Modification and Re-Routing

With importance predictions in hand—whether from the ground-truth method, an LLM, or a baseline—the next step is to feed them back into the routing pipeline:

1. For each method $m \in \{\text{GT}, \text{LLM}, \text{Betweenness}, \text{Degree}\}$, construct modified edge costs

$$c'_e = c_e \left(1 + \alpha \hat{I}_e^{(m)} \right).$$

2. Re-solve the TSP on the modified graph to obtain a method-specific route R_m and cost C_m .
3. Record both the route and cost for downstream evaluation (Section 6.3).

Warm-starting from R_{base} is again recommended to control solver variance. The ground-truth variant (method GT) uses I_e directly and defines the ceiling for routing quality improvement; all other methods are measured relative to it.

6.2.2 Normalization Strategy

A persistent scaling issue arises because different predictors output importance values on incompatible scales. The current candidate—sum normalisation—has the advantage of simplicity but is sensitive to outliers. Future work should compare:

- **Sum normalisation:** $\hat{I}_e \leftarrow \hat{I}_e / \sum_{e'} \hat{I}_{e'}$.
- **Min-max normalisation:** maps all scores to $[0, 1]$.
- **Rank normalisation:** replaces raw scores with their fractional ranks, eliminating scale entirely and matching the rank-based evaluation metrics discussed in Section 6.3.

Rank normalisation is particularly attractive because it decouples the predictor from the choice of α , and Kendall's τ / Spearman ρ are invariant to any monotone transformation of the scores anyway.

6.3 Evaluation

6.3.1 Importance Prediction Quality

Two complementary metrics assess how well $\hat{I}_e$ approximates I_e :

- **Spearman Rank Correlation (ρ).** Measures monotone agreement between predicted and ground-truth importance rankings across all edges in R_{base} . A value near +1 indicates the predictor perfectly reproduces the ground-truth ordering.
- **Top- K Overlap.** Computes the fraction of the top- K most-important edges (by ground truth) that also appear in the top- K predicted edges. Using $K = \lfloor 0.1 \times |R_{\text{base}}| \rfloor$ (approximately the top 10%) focuses attention on the critical edges that matter most for route modification.

If scaling inconsistencies prove problematic even after normalisation (Section 6.2.2), Kendall's τ provides an alternative rank-based metric that is more robust to tied scores.

6.3.2 Routing Quality

Even a predictor with mediocre Spearman correlation may produce useful routes if the errors are concentrated on unimportant edges. We therefore assess routing quality directly via:

- **Tour cost comparison.**

$$\Delta C_m = C_m - C_{\text{GT}},$$

where smaller (or negative) values indicate the method routes as well as (or better than) the ground-truth-perturbed solution. Reporting $C_m/C(G)$ also contextualises the absolute deviation.

- **Path similarity (edge overlap).** The Jaccard coefficient between the edge sets of R_m and R_{GT} :

$$J(R_m, R_{\text{GT}}) = \frac{|E(R_m) \cap E(R_{\text{GT}})|}{|E(R_m) \cup E(R_{\text{GT}})|}.$$

High overlap indicates the method arrives at a structurally similar detour around high-importance edges, even if the exact cost differs.

6.3.3 Hyper-Parameter Sensitivity (α Tuning)

The cost-inflation weight α jointly affects all methods and may interact differently with each predictor’s score distribution. Future experiments should:

1. Sweep $\alpha \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$ for each predictor.
2. Report ΔC_m and $J(R_m, R_{\text{GT}})$ as a function of α .
3. Identify whether a single α^* transfers across instances or whether per-predictor tuning is necessary.

6.4 Scaling and Long-Term Directions

6.4.1 Large-Scale Batch Processing

The current implementation processes a single route at a time. Productionising the framework for Amazon’s full delivery network requires:

- **Multi-route parallelism.** While individual edge perturbations are parallelised within a route, the outer loop over routes can be distributed across worker nodes using a job queue (e.g., AWS Batch or Ray).
- **Incremental caching.** Store intermediate results (perturbed costs, base routes) so that re-computation is triggered only when the underlying distance matrix changes.
- **Solver warm-starting at scale.** Investigate whether the warm-start speedup ($\approx 3\times$) holds as instance size grows beyond 200 nodes, and whether population-based initialisations (e.g., from a pool of diverse prior solutions) improve it further.

6.4.2 Learning-Based Predictors

Beyond LLMs, several neural architectures are natural candidates for importance prediction from graph structure:

- **Graph Neural Networks (GNNs).** Models such as Graph Attention Networks (GAT) or Graph Convolutional Networks (GCN) can aggregate neighbourhood information and output per-edge importance scores. Training on a corpus of (route, ground-truth importance) pairs would allow zero-shot generalisation to unseen instances.
- **Attention-based sequence models.** If edges are serialised by their position in R_{base} , a Transformer encoder can learn positional importance patterns (e.g., edges leaving sparse sub-regions tend to be critical).
- **Hybrid models.** Combine structural features (betweenness, degree) with learned embeddings as input to a lightweight regression head.

6.4.3 Time-Dependent TSP

Real last-mile delivery is subject to time windows and traffic variability. Extending the framework to handle time-dependent travel times would require:

- Re-defining edge importance with respect to a time-expanded graph, where each edge (u, v, t) carries a travel time that depends on departure time t .
- Adapting the perturbation procedure to penalise all temporal copies of an edge simultaneously.
- Evaluating whether LLM-based predictors can incorporate time-window metadata in their prompts to produce temporally-aware importance estimates.

6.4.4 Generalisation Across Instance Types

The current evaluation is limited to a single 119-node instance. A rigorous generalisation study should vary:

- **Instance size:** 50, 100, 200, 500 nodes.
- **Graph density:** $K \in \{2, 4, 6, 8\}$ for the KNN construction.
- **Geographic spread:** dense urban grids vs. sparse suburban/rural networks, where bottleneck edges are structurally different.
- **Solver:** compare LKH-3 results against OR-Tools to assess whether importance scores are solver-agnostic.

6.4.5 Downstream Applications

Edge importance scores have value beyond evaluating predictors. Promising downstream uses include:

- **Graph pruning.** Remove low-importance edges from the KNN graph before solving the TSP, reducing problem size without significantly degrading solution quality.
- **Robust routing.** Proactively inflate the cost of high-importance edges to produce routes that are less sensitive to real-time disruptions (road closures, traffic incidents).
- **Driver guidance.** Flag critical turn or road segments to human drivers as segments where deviation is particularly costly.

7 Conclusion

We have successfully implemented and validated the ground-truth edge importance framework for TSP instances derived from real-world last-mile delivery data. The framework computes exact importance scores via parallel solver invocations with warm-starting, enabling efficient evaluation on instances with 100+ nodes. The resulting importance scores provide a critical benchmark for training and validating learning-based edge importance predictors, laying the groundwork for intelligent routing systems that can predict which edges are critical and adapt routes accordingly.

The modular, production-ready implementation in Python uses only standard library constructs and integrates seamlessly with the LKH-3 solver, making it portable across systems and extensible for future variants.

References

- [1] D. Merchán, J. Arora, J. Pachon, K. Konduri, M. Winkenbach, S. Parks, and J. Noszek, “2021 Amazon Last Mile Routing Research Challenge: Data Set,” *Transportation Science*, vol. 56, no. 6, pp. 1452–1472, 2022.
- [2] K. Helsgaun, “LKH: An Effective Implementation of the Lin-Kernighan Heuristic for the TSP,” *European J. Operational Research*, vol. 126, no. 3, pp. 541–549, 2000.
- [3] S. Lin and B. W. Kernighan, “An Effective Heuristic Algorithm for the Traveling-Salesman Problem,” *Operations Research*, vol. 21, no. 2, pp. 498–516, 1973.
- [4] E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan, and D. B. Shmoys, *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. John Wiley & Sons, 1985.
- [5] K. Clarkson, “Fast Algorithms for the Uniform Random Projection,” *Proc. 19th Annu. Symp. Computational Geometry*, pp. 172–181, ACM, 2003.
- [6] U. Brandes, “A Faster Algorithm for Betweenness Centrality,” *Journal of Mathematical Sociology*, vol. 25, no. 2, pp. 163–177, 2001.
- [7] R. Gendreau, A. Hertz, and G. Laporte, “A Tabu Search Algorithm for the Vehicle Routing Problem,” *Management Science*, vol. 40, no. 10, pp. 1276–1290, 1994.